

Lesson 9: Authentication & Authorization

📖 Table of Contents

1. [Introduction to Authentication](#introduction-to-authentication)
2. [Laravel Breeze Setup](#laravel-breeze-setup)
3. [Authentication System](#authentication-system)
4. [Authorization & Gates](#authorization--gates)
5. [Policies](#policies)
6. [Middleware Authentication](#middleware-authentication)
7. [Password Management](#password-management)
8. [Remember Me & Sessions](#remember-me--sessions)
9. [API Token Authentication](#api-token-authentication)
10. [Practical Examples](#practical-examples)

Introduction to Authentication

What is Authentication?

****Authentication**** = Verifying user identity (Who are you?)

****Authorization**** = Verifying user permissions (What can you do?)

...

Login

 ← Authentication (Verify Identity)

↓

Dashboard

 ← Authorization (Check Permissions)

...

Difference between Authentication and Authorization

Authentication	Authorization
Who are you?	What can you do?
Verify login credentials	Check permissions
Login/Register	Permissions/Roles
<code>`auth()->check()`</code>	<code>`Gate::allows()`</code>

Laravel Breeze Setup

What is Laravel Breeze?

****Laravel Breeze**** = A simple and fast authentication system ready to use, including:

- Login, Register, Logout
- Password Reset
- Email Verification
- Profile Management

Installation

```
```bash
```

```
1. Install Breeze
```

```
composer require laravel/breeze --dev
```

```
2. Install Scaffolding
```

```
php artisan breeze:install blade
```

```
3. Install NPM Dependencies
```

```
npm install && npm run dev
```

```
4. Run Migrations
```

```
php artisan migrate
```

```
```
```

Generated Files

```
```
```

```
app/
```

```
├─ Http/Controllers/Auth/
```

```
| ├─ AuthenticatedSessionController.php # Login
```

```
| ├─ RegisteredUserController.php # Register
```

```
| ├─ PasswordResetLinkController.php # Forgot Password
```

```
| └─ ...
```

```
resources/
```

```
├─ views/auth/
```

```
| ├─ login.blade.php
```

```
| ├─ register.blade.php
```

```
| └─ ...
```

```
routes/
```

```
├─ auth.php # Authentication Routes
```

```
```
```

```
---
```

Authentication System

Login

****AuthenticatedSessionController.php:****

```php

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Auth;

class AuthenticatedSessionController extends Controller

{

/\*\*

\* Display login page

\*/

public function create()

{

return view('auth.login');

}

/\*\*

\* Handle login

\*/

public function store(Request \$request)

{

\$credentials = \$request->validate([

'email' => 'required|email',

'password' => 'required',

]);

if (Auth::attempt(\$credentials, \$request->boolean('remember'))) {

\$request->session()->regenerate();

return redirect()->intended('dashboard');

}

return back()->withErrors([

'email' => 'Invalid credentials',

])->onlyInput('email');

}

/\*\*

\* Logout

\*/

public function destroy(Request \$request)

{

Auth::guard('web')->logout();

```

 $request->session()->invalidate();
 $request->session()->regenerateToken();

 return redirect('/');
 }
}
...

Register

RegisteredUserController.php:
```php
use App\Models\User;
use Illuminate\Support\Facades\Hash;
use Illuminate\Auth\Events\Registered;

class RegisteredUserController extends Controller
{
    public function create()
    {
        return view('auth.register');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect('dashboard');
    }
}
...

```

Checking User Authentication

```
```php
// Check if logged in
if (auth()->check()) {
 // User is authenticated
}

// Check if guest
if (auth()->guest()) {
 // User is not authenticated
}

// Get current user
$user = auth()->user();
$user = Auth::user();

// Get user ID
$id = auth()->id();

// In Blade
@auth

Welcome {{ auth()->user()->name }}

@endauth

@guest
Login
@endguest
```
```

Manual Login

```
```php
use Illuminate\Support\Facades\Auth;

// Login with Credentials
if (Auth::attempt(['email' => $email, 'password' => $password])) {
 // Success
}

// Login with Remember Me
if (Auth::attempt($credentials, $remember = true)) {
 // Success
}
```

```

}

// Login with User Model directly
Auth::login($user);
Auth::login($user, $remember = true);

// Login for single request only
Auth::once($credentials);

// Login by ID
Auth::loginUsingId(1);

// Logout
Auth::logout();
```

---

## Authorization & Gates

### What are Gates?

**Gates** = A simple way to check permissions

### Defining Gates

**app/Providers/AppServiceProvider.php:**
```php
use Illuminate\Support\Facades\Gate;

public function boot(): void
{
 // Simple Gate
 Gate::define('update-post', function (User $user, Post $post) {
 return $user->id === $post->user_id;
 });

 // Admin Gate
 Gate::define('access-admin', function (User $user) {
 return $user->role === 'admin';
 });

 // Gate with before check (Admin can do everything)
 Gate::before(function (User $user, string $ability) {
 if ($user->role === 'super-admin') {
 return true;
 }
 })
}

```

```

 });
}
...

Using Gates

```php
use Illuminate\Support\Facades\Gate;

// In Controller
public function update(Request $request, Post $post)
{
    if (Gate::allows('update-post', $post)) {
        // Can update
        $post->update($request->validated());
    }

    if (Gate::denies('update-post', $post)) {
        abort(403);
    }

    // Or use authorize (throws 403 automatically)
    Gate::authorize('update-post', $post);

    $post->update($request->validated());
}

// In Blade
@can('update-post', $post)
    Edit
@endcan

@cannot('update-post', $post)

You cannot edit this post

@endcannot

// Using User Model
if ($user->can('update-post', $post)) {
    //
}

if ($user->cannot('update-post', $post)) {
    //
}

```

```

}
...

### Advanced Gates

```php
// Gate with multiple Parameters
Gate::define('update-comment', function (User $user, Post $post, Comment $comment) {
 return $user->id === $comment->user_id
 && $comment->post_id === $post->id;
});

// Gate without User (for Guests)
Gate::define('view-post', function (?User $user, Post $post) {
 if ($post->is_published) {
 return true;
 }

 return $user && $user->id === $post->user_id;
});

// Usage
Gate::authorize('update-comment', [$post, $comment]);
...

Policies

What are Policies?

Policy = An organized class for permissions related to a specific Model

Creating a Policy

```bash
# Create Policy
php artisan make:policy PostPolicy

# Create Policy with Model
php artisan make:policy PostPolicy --model=Post
...

**app/Policies/PostPolicy.php:**
```php
is_published) {
 return true;
}

```



```

 }

 // Only post owner can see drafts
 return $user && $user->id === $post->user_id;
}

/**
 * Create new post
 */
public function create(User $user): bool
{
 return $user->email_verified_at !== null;
}

/**
 * Update post
 */
public function update(User $user, Post $post): bool
{
 return $user->id === $post->user_id;
}

/**
 * Delete post
 */
public function delete(User $user, Post $post): bool
{
 return $user->id === $post->user_id;
}

/**
 * Restore deleted post
 */
public function restore(User $user, Post $post): bool
{
 return $user->id === $post->user_id;
}

/**
 * Permanently delete post
 */
public function forceDelete(User $user, Post $post): bool
{
 return $user->role === 'admin';
}
}
...

```

### ### Registering Policy

**\*\*app/Providers/AppServiceProvider.php:\*\***

```
```php
use App\Models\Post;
use App\Policies\PostPolicy;
use Illuminate\Support\Facades\Gate;

public function boot(): void
{
    Gate::policy(Post::class, PostPolicy::class);
}
```
```

Or use Auto-Discovery (Laravel discovers automatically):

```
```
app/Models/Post.php → app/Policies/PostPolicy.php
app/Models/Comment.php → app/Policies/CommentPolicy.php
```
```

### ### Using Policies

```
```php
// In Controller
public function update(Request $request, Post $post)
{
    // Method 1: authorize() (throws 403 automatically)
    $this->authorize('update', $post);

    $post->update($request->validated());
}

public function destroy(Post $post)
{
    // Method 2: Gate::authorize()
    Gate::authorize('delete', $post);

    $post->delete();
}

public function show(Post $post)
{
    // Method 3: can()
    if (auth()->user()->can('view', $post)) {
        return view('posts.show', compact('post'));
    }
}
```
```

```
 abort(403);
 }
}
```

```
// In Blade
@can('update', $post)
```

[Edit](#)

```
@endcan
```

```
@can('delete', $post)
```

```
 @csrf
```

```
 @method('DELETE')
```

Delete

```
@endcan
```

```
// In Route
```

```
Route::put('/posts/{post}', [PostController::class, 'update'])
```

```
 ->can('update', 'post'); // 'post' = route parameter name
```

```
...
```

### ### Advanced Policy Methods

```
```php
```

```
class PostPolicy
```

```
{
```

```
    /**
```

```
     * Before all checks (Admin can do everything)
```

```
     */
```

```
    public function before(User $user, string $ability): ?bool
```

```
    {
```

```
        if ($user->role === 'super-admin') {
```

```
            return true;
```

```
        }
```

```
        return null; // Continue normal checks
```

```
    }
```

```
    /**
```

```
     * Update post with additional conditions
```

```
     */
```

```
    public function update(User $user, Post $post): bool
```

```
    {
```

```
        // Post owner
```

```

        if ($user->id === $post->user_id) {
            return true;
        }

        // Moderator if not published
        if ($user->role === 'moderator' && !$post->is_published) {
            return true;
        }

        return false;
    }
}
...

---

## Middleware Authentication

### auth Middleware

```php
// In routes/web.php
Route::middleware('auth')->group(function () {
 Route::get('/dashboard', [DashboardController::class, 'index']);
 Route::resource('posts', PostController::class);
});

// Or in Controller
class PostController extends Controller
{
 public function __construct()
 {
 $this->middleware('auth');

 // Or for specific actions
 $this->middleware('auth')->only(['create', 'store', 'edit', 'update']);
 $this->middleware('auth')->except(['index', 'show']);
 }
}
...

guest Middleware

```php
// For pages that only guests can access
Route::middleware('guest')->group(function () {
    Route::get('/login', [LoginController::class, 'create']);
});

```

```

Route::post('/login', [LoginController::class, 'store']);
Route::get('/register', [RegisterController::class, 'create']);
Route::post('/register', [RegisterController::class, 'store']);
});
...

### verified Middleware

```php
// For pages requiring email verification
Route::middleware(['auth', 'verified'])->group(function () {
 Route::get('/premium', [PremiumController::class, 'index']);
});
...

Custom Auth Middleware

```bash
php artisan make:middleware EnsureUserIsAdmin
...

**app/Http/Middleware/EnsureUserIsAdmin.php:**
```php
check() || auth()->user()->role !== 'admin') {
 abort(403, 'Unauthorized');
}

 return $next($request);
}
}
...

bootstrap/app.php or app/Http/Kernel.php:
```php
protected $middlewareAliases = [
    // ...
    'admin' => \App\Http\Middleware\EnsureUserIsAdmin::class,
];
...

**Usage:**
```php
Route::middleware('admin')->group(function () {
 Route::get('/admin/dashboard', [AdminController::class, 'index']);
});
...

```

---

## ## Password Management

### ### Password Reset

Laravel Breeze provides a password reset system:

```

1. User requests reset → sends email
2. User clicks link in email
3. User enters new password
4. Password is updated

```

### ### Changing Password

```php

```
use Illuminate\Support\Facades\Hash;
```

```
public function updatePassword(Request $request)
```

```
{
```

```
    $request->validate([
        'current_password' => 'required|current_password',
        'password' => 'required|string|min:8|confirmed',
    ]);
```

```
    $request->user()->update([
        'password' => Hash::make($request->password),
    ]);
```

```
    return back()->with('status', 'Password updated successfully');
```

```
}
```

```

### ### Verifying Password

```php

```
use Illuminate\Support\Facades\Hash;
```

```
// Check
```

```
if (Hash::check('plain-text-password', $hashedPassword)) {
```

```
    // Correct
```

```
}
```

```
// In Validation
```

```
$request->validate([
```

```

        'current_password' => 'required|current_password',
    ]);
    ...

    ### Password Hashing

    ```php
 use Illuminate\Support\Facades\Hash;

 // Hashing
 $hashed = Hash::make('password');

 // Check if needs rehash
 if (Hash::needsRehash($hashed)) {
 $hashed = Hash::make('password');
 }
 ...

 ## Remember Me & Sessions

 ### Remember Me

    ```php
    // Login with Remember Me
    if (Auth::attempt($credentials, $remember = true)) {
        // Stays logged in for 5 years
    }

    // In Form
    ☐
    Remember Me
    ...

    ### Session Management

    ```php
 // Regenerate Session (prevent Session Fixation)
 $request->session()->regenerate();

 // Invalidate Session
 $request->session()->invalidate();

 // Regenerate CSRF Token
 $request->session()->regenerateToken();

```

```

// On Logout - do all three
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
...

Session Configuration

config/session.php:
```php
return [
    'lifetime' => 120,           // Session lifetime in minutes
    'expire_on_close' => false,  // Session expires on browser close
    'encrypt' => false,         // Encrypt session
    'driver' => env('SESSION_DRIVER', 'file'),
];
```

API Token Authentication

Laravel Sanctum

```bash
# Install Sanctum
composer require laravel/sanctum

# Publish Migration
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"

# Run Migration
php artisan migrate
```

Creating Tokens

User Model:
```php
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
```

```



**\*\*Create Token:\*\***

```
```php
```

```
// Simple token
```

```
$token = $user->createToken('token-name')->plainTextToken;
```

```
// Token with Abilities (permissions)
```

```
$token = $user->createToken('token-name', ['posts:create', 'posts:update'])  
    ->plainTextToken;
```

```
```
```

**### Using Tokens**

```
```php
```

```
// In routes/api.php
```

```
Route::middleware('auth:sanctum')->group(function () {  
    Route::get('/user', function (Request $request) {  
        return $request->user();  
    });  
});
```

```
Route::apiResource('posts', PostController::class);  
});  
```
```

**\*\*API Request:\*\***

```
```bash
```

```
curl -H "Authorization: Bearer {token}" \  
    -H "Accept: application/json" \  
    https://example.com/api/user  
```
```

**### Checking Abilities**

```
```php
```

```
// In Controller
```

```
if ($request->user()->tokenCan('posts:create')) {  
    // Can create post  
}
```

```
// Middleware
```

```
Route::post('/posts', [PostController::class, 'store'])  
    ->middleware('auth:sanctum', 'abilities:posts:create');  
```
```

```

```

**## Practical Examples**

### ### Example 1: Blog System with Permissions

**\*\*PostPolicy.php:\*\***

```php

class PostPolicy

{

public function viewAny(?User \$user): bool

{

return true; // Everyone can view list

}

public function view(?User \$user, Post \$post): bool

{

if (\$post->is_published) {

return true;

}

return \$user && \$user->id === \$post->user_id;

}

public function create(User \$user): bool

{

return \$user->email_verified_at !== null;

}

public function update(User \$user, Post \$post): bool

{

return \$user->id === \$post->user_id;

}

public function delete(User \$user, Post \$post): bool

{

return \$user->id === \$post->user_id

|| \$user->role === 'admin';

}

}

```

**\*\*PostController.php:\*\***

```php

class PostController extends Controller

{

public function __construct()

{

\$this->middleware('auth')->except(['index', 'show']);

}

```
public function index()
{
    $this->authorize('viewAny', Post::class);

    $posts = Post::where('is_published', true)
        ->latest()
        ->paginate(10);

    return view('posts.index', compact('posts'));
}

public function show(Post $post)
{
    $this->authorize('view', $post);

    return view('posts.show', compact('post'));
}

public function create()
{
    $this->authorize('create', Post::class);

    return view('posts.create');
}

public function store(StorePostRequest $request)
{
    $this->authorize('create', Post::class);

    $post = auth()->user()->posts()->create($request->validated());

    return redirect()->route('posts.show', $post);
}

public function edit(Post $post)
{
    $this->authorize('update', $post);

    return view('posts.edit', compact('post'));
}

public function update(UpdatePostRequest $request, Post $post)
{
    $this->authorize('update', $post);

    $post->update($request->validated());
}
```

```

        return redirect()->route('posts.show', $post);
    }

    public function destroy(Post $post)
    {
        $this->authorize('delete', $post);

        $post->delete();

        return redirect()->route('posts.index');
    }
}
```

```

### ### Example 2: Roles & Permissions System

**\*\*User Model:\*\***

```

```php
class User extends Authenticatable
{
    public function hasRole(string $role): bool
    {
        return $this->role === $role;
    }

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }

    public function isModerator(): bool
    {
        return $this->role === 'moderator';
    }

    public function canModerate(): bool
    {
        return in_array($this->role, ['admin', 'moderator']);
    }
}
```

```

**\*\*Gates:\*\***

```

```php
// In AppServiceProvider
Gate::define('manage-users', function (User $user) {
    return $user->isAdmin();
});

```

```

});

Gate::define('moderate-posts', function (User $user) {
    return $user->canModerate();
});

Gate::define('edit-comments', function (User $user, Comment $comment) {
    return $user->id === $comment->user_id
        || $user->canModerate();
});
...

**Middleware:**
```php
class CheckRole
{
 public function handle(Request $request, Closure $next, string $role)
 {
 if (!auth()->check() || !auth()->user()->hasRole($role)) {
 abort(403);
 }

 return $next($request);
 }
}
...

Routes:
```php
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('/admin/dashboard', [AdminController::class, 'index']);
    Route::resource('/admin/users', UserController::class);
});

Route::middleware(['auth', 'role:moderator'])->group(function () {
    Route::get('/moderate/posts', [ModerateController::class, 'posts']);
});
...

```

Example 3: API Authentication

```

**AuthController.php:**
```php
class AuthController extends Controller
{
 public function register(Request $request)
 {

```

```

$request->validate([
 'name' => 'required|string|max:255',
 'email' => 'required|email|unique:users',
 'password' => 'required|string|min:8|confirmed',
]);

$user = User::create([
 'name' => $request->name,
 'email' => $request->email,
 'password' => Hash::make($request->password),
]);

$token = $user->createToken('auth-token')->plainTextToken;

return response()->json([
 'user' => $user,
 'token' => $token,
], 201);
}

public function login(Request $request)
{
 $request->validate([
 'email' => 'required|email',
 'password' => 'required',
]);

 if (!Auth::attempt($request->only('email', 'password'))) {
 return response()->json([
 'message' => 'Invalid credentials'
], 401);
 }

 $user = User::where('email', $request->email)->firstOrFail();

 $token = $user->createToken('auth-token')->plainTextToken;

 return response()->json([
 'user' => $user,
 'token' => $token,
]);
}

public function logout(Request $request)
{
 $request->user()->currentAccessToken()->delete();
}

```

```

 return response()->json([
 'message' => 'Logged out successfully'
]);
 }

 public function me(Request $request)
 {
 return response()->json($request->user());
 }
}
...

routes/api.php:
```php
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);

    Route::apiResource('posts', PostController::class);
});
...

```

Best Practices

1. Use Policies for Complex Logic

```

```php
// Good - organized
$this->authorize('update', $post);

// Bad - logic in Controller
if (auth()->user()->id !== $post->user_id) {
 abort(403);
}
...

```

### ### 2. Regenerate Session on Login

```

```php
// Prevent Session Fixation Attack
$request->session()->regenerate();
...

```

3. Always Use Hash::make()

```
```php
// Safe
Hash::make($password)

// Dangerous - don't do this
bcrypt($password) // Old
password_hash() // Don't use
```
```

4. Verify Email

```
```php
Route::middleware(['auth', 'verified'])->group(function () {
 // routes
});
```
```

Common Mistakes

1. Forgetting authorize()

```
```php
// Dangerous
public function update(Request $request, Post $post)
{
 $post->update($request->validated());
}

// Safe
public function update(Request $request, Post $post)
{
 $this->authorize('update', $post);
 $post->update($request->validated());
}
```
```

2. Using auth()->user() without checking

```
```php
// Dangerous - may be null
$userId = auth()->user()->id;
```



```
// Safe
if (auth()->check()) {
 $userId = auth()->user()->id;
}
```

```
// Or
$userId = auth()->id();
```
```

3. Not invalidating Session on Logout

```
```php
```

```
// Incomplete
Auth::logout();
```

```
// Complete
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
```
```

Next Steps

After completing this lesson, you're ready for:

****Lesson 10****: File Upload & Storage

- File Upload
- Storage Configuration
- Image Processing

****Happy Learning!****